Esercitazione di Laboratorio Informatica@DSS (2022/2023)

Paolo Franciosa

None

Lo scopo del laboratorio è di esercitarsi e misurare la propria preparazione: gli esercizi non sono troppo difficili, se si sono seguite le lezioni. Non vi viene comunque messo alcun voto.

Modalità di lavoro: gli studenti devono lavorare in autonomia o in piccoli gruppi, senza disturbare i colleghi. Il lavoro di gruppo è fruttuoso solo se tutti partecipano e se ognuno scrive una propria soluzione per tutti gli esercizi.

Il docente cercherà per quanto possibile di non occupare il tempo del laboratorio per introdurre materiale nuovo, anche se a volte questo sarà necessario. Il docente è a disposizione per aiutare gli studenti, che possono iniziare a lavorare anche prima che il docente arrivi in aula, se lo desiderano

Raccomandazioni: leggete bene il testo degli esercizi prima di chiedere chiarimenti. In ogni caso sarò in aula con voi.

Uso dei file di test: per aiutarvi a completare questa esercitazione avete a disposizione dei programmi di test per testare la vostra soluzione. Questi sono simili a quelli che avrete in sede di esame, pertanto vi consiglio di imparate ad usarli. Per portare a termine l'esercizio è necessario

- scrivere un file di soluzione **col nome specificato**;
- avere dentro la funzione **col nome specificato**;
- porre il file di test corrispondente **nella stessa cartella**;
- eseguire in quella cartella il comando `python3 <fileditest>`

dove `<fileditest>` va ovviamente sostituito con il nome del file di test appropriato per l'esercizio su cui state lavorando. Per ogni esercizio ci sta un file di test indipendente, così da poter lavorare sugli esercizi uno alla volta con più agio.

Il risultato di ogni test è una schermata (o più schermate) nella quale si mostra:

- se è stato trovato il file con il nome corretto,
- se il file contiene la funzione con il nome corretto,
- se chiamate alla funzione con diversi valori dei parametri terminano restituendo risultati corretti.

Per ogni funzione scritta vengono eseguite chiamate con diversi valori dei parametri. L'esito dei test viene riportato con il carattere

- `E` se la chiamata non può essere eseguita,
- `F` se la funzione non restituisce il risultato corretto,
- `.` se la funzione restituisce il risultato corretto.

1 Conteggio delle vocali

Scrivete una funzione `conteggiovocali(testo)` che restituisca il numero di vocali presenti nella stringa `testo`. Ad esempio

```
print(conteggiovocali("Contate le vocali."))
```

```
7
```

Il programma Python deve essere salvato nel file: `conteggiovocali.py`

File di test: `test_conteggiovocali.py`

2 Spazi a capo

Scrivete una funzione `spaziacapo(s)` che sostituisca ogni spazio di `s` con un simbolo a capo `\n`. Ad esempio

```
print(spaziacapo("c'era una volta"))
```

```
c'era
una
volta
```

Il programma Python deve essere salvato nel file: `spaziacapo.py`

File di test: `test_spaziacapo.py`

3 Moltiplica caratteri

Scrivete una funzione `moltiplicacaratteri(s, c, x)` che sostituisca ogni occorrenza di `c` con `x` occorrenze consecutive. Ad esempio

```
print(moltiplicacaratteri('abcb','b',3))
```

```
abbbcbbb
```

Il programma Python deve essere salvato nel file: `moltiplicacaratteri.py`

File di test: `test_moltiplicacaratteri.py`

4 Scambia caratteri

Scrivi una funzione `scambia_caratteri(s, x, y)` che prenda in input - una stringa `s` - due interi `x` e `y`

La funzione deve restituire una stringa uguale a `s` ma con i caratteri in posizione `x` e `y` scambiati. Se `x` o `y` non sono posizioni valide, la funzione deve restituire `None`.

Ad esempio

```
print(scambia_caratteri('balletto', 1, 7))
```

```
bolletta
```

Il programma Python deve essere salvato nel file: `scambia_caratteri.py`

File di test: `test_scambia_caratteri.py`

5 Rimuovi Doppie

Scrivete una funzione `rimuoviDoppie(s)` che prende una stringa `s` in input e restituisce una stringa uguale, ma priva di doppie. Ad esempio

```
print(rimuoviDoppie('tapparella'))
```

```
taparela
```

Il programma Python deve essere salvato nel file: `rimuoviDoppie.py`

File di test: `test_rimuoviDoppie.py`

6 Rimuovi Ripetizioni

Scrivete una funzione `rimuoviripetizioni(s)` che prende una stringa `s` in input e restituisce una stringa uguale, ma priva di ripetizioni. Ad esempio

```
print(rimuoviripetizioni('allietare'))
```

```
alietr
```

Il programma Python deve essere salvato nel file: `rimuoviripetizioni.py`

File di test: `test_rimuoviripetizioni.py`

7 Formatta secondi

Scrivere una funzione `formatta_hhssmm(sec)` che:

- dato in input un numero di secondi `sec`;
- restituisca l'equivalente in formato hh:mm:ss;
- le ore, i minuti e i secondi devono essere stampati sempre con almeno due cifre, ad esempio 9 è 09 e 7 è 07 e 0 è 00.
- le ore possono utilizzare un numero arbitrario di cifre (ma sempre almeno due).

Scrivere a tal fine le seguenti funzioni ausiliarie - una funzione che dato un numero di secondi totali restituisce tre numeri corrispondenti a ore, minuti e secondi. - un'altra funzione per gestire la conversione da numero a stringa, con in più l'aggiunta dello zero iniziale, quando questo zero iniziale dovesse servire.

A tal fine, consultare https://pyformat.info

Esempi: - `formatta_hhssmm(11537)` ritorna il valore `03:12:17`

La funzione deve essere salvata in un file che si chiama `formatta_hhssmm.py`

Il programma Python deve essere salvato nel file: `formatta_hhssmm.py`

File di test: `test_formatta_hhssmm.py`